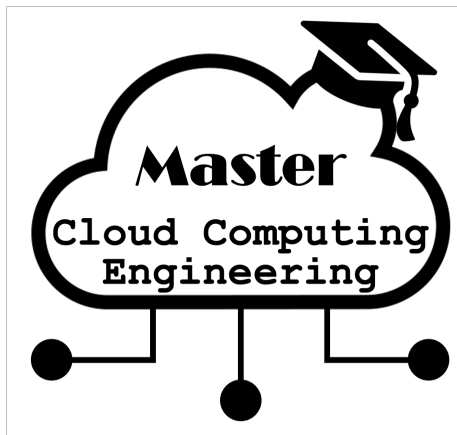




DevOps PT - Assignment
Release/Deploy/Operate - Mika

Hochschule Burgenland
MCCE24
Sommersemester 2025

Harald Beier* Susanne Peer† Patrick Prugger‡
Philipp Palatin§
22. Juni 2025

*2410781028@hochschule-burgenland.at

†2410781002@hochschule-burgenland.at

‡2410781029@hochschule-burgenland.at

§2310781027@hochschule-burgenland.at

Inhaltsverzeichnis

1	Einleitung	3
2	Technologieauswahl	3
2.1	Terraform als Provisionierungstool	3
2.2	Architekturübersicht	3
3	Umsetzungskonzept	4
3.1	Authentifizierung in AWS	4
3.2	Provisionierungs-Workflow	5
4	Vor-/Herausforderungen	5
4.1	Vorteile	5
4.2	Herausforderungen	5
5	Technische Umsetzung	6
5.1	Terraform-Modulstruktur	6
5.2	Beispiel: DynamoDB-Provisionierung	6
	Literaturverzeichnis	7

1 Einleitung

Dieses Dokument beschreibt die Planung, wie eine To-Do-App in der AWS-Cloud automatisch bereitgestellt werden kann. Das Ziel ist es alle nötigen Komponenten automatisch über Infrastructure-as-Code (IaC) bereitzustellen. Die Lösung besteht aus Benutzerschnittstelle, Backend, Datenbank und Sicherheitskomponenten.

2 Technologieauswahl

2.1 Terraform als Provisionierungstool

Als primäres Provisionierungstool wurde Terraform gewählt, da es Plattformunabhängigkeit ermöglicht und eine Multi-Cloud Strategie unterstützt [HashiCorp, 2025f]. Die deklarative Syntax in Form von HashiCorp Configuration Language (HCL) sorgt für gut lesbare und wartbare Konfigurationen [HashiCorp, 2025a]. Das integrierte State-Management erlaubt die Nachverfolgung aller Ressourcen [?]. Dank der großen Community und der umfangreichen AWS-Modulbibliothek [HashiCorp, 2025e] lassen sich konsistente Architekturen über verschiedene Umgebungen hinweg realisieren. Als Alternativen wurden AWS CloudFormation [Amazon Web Services, 2025a] und Pulumi [Pulumi, 2025] evaluiert, jedoch aufgrund von Vendor-Lock-in bzw. anderer Architekturansätze verworfen.

2.2 Architekturübersicht

Die Architektur der To-Do-Applikation (vgl. Abbildung 1) basiert auf einer Serverless-Architektur in der AWS-Cloud. Das statische Frontend wird in einem S3-Bucket gehostet und weltweit über CloudFront als Content Delivery Network (CDN) ausgeliefert. API-Anfragen gelangen von CloudFront zum API Gateway, das als Schnittstelle zwischen Frontend und Backend dient [Amazon Web Services, 2024]. Die Backend-Logik ist in Lambda-Funktionen implementiert, welche die Geschäftslogik abbilden und mit DynamoDB für die Speicherung der To-Do-Items interagieren [?]. Die Benutzerauthentifizierung und -verwaltung erfolgt zentral über AWS Cognito.

Die Kommunikation zwischen den Komponenten folgt einem klar definierten Fluss, hierbei dient CloudFront als zentraler Einstiegspunkt und leitet statische Inhalte an den S3-Bucket sowie API-Aufrufe an das API Gateway weiter. Das API Gateway wiederum leitet die Anfragen an die entsprechenden Lambda-Funktionen weiter, die für die Geschäftslogik zuständig sind. Die Lambda-Funktionen interagieren mit DynamoDB für CRUD-Operationen und mit Cognito für die Authentifizierung. Der S3-Bucket kommuniziert direkt mit Cognito für den Login-Prozess.

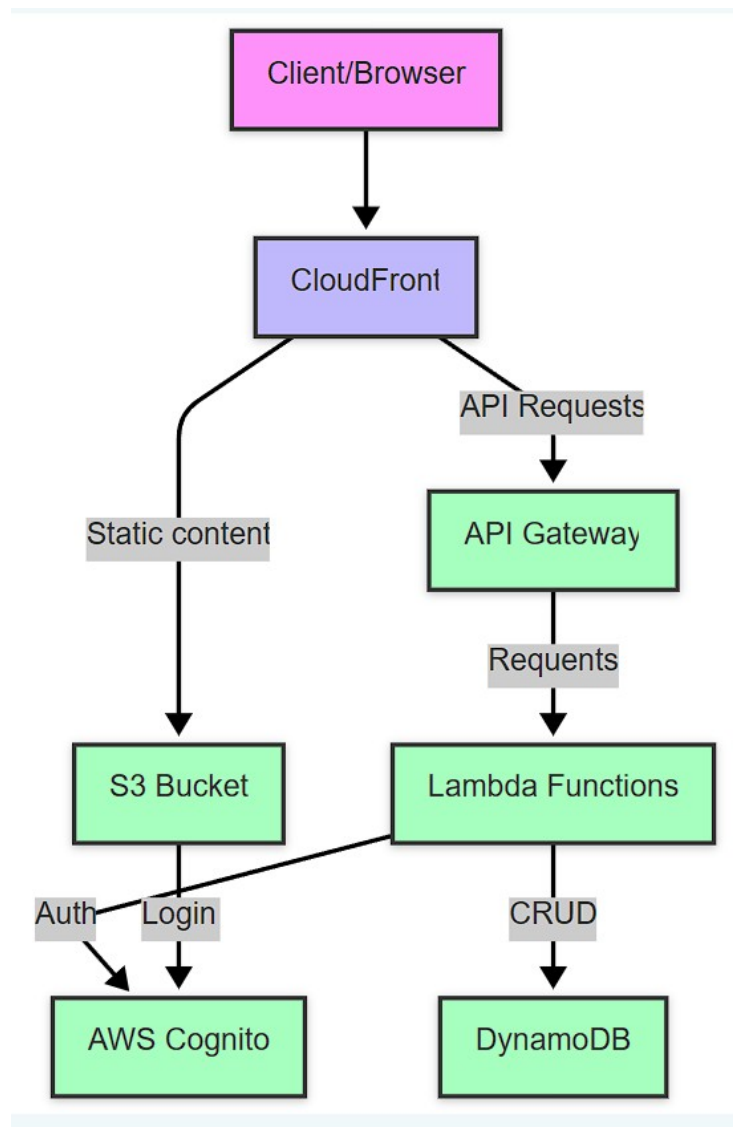


Abbildung 1: Architekturübersicht der To-Do-Applikation

3 Umsetzungskonzept

3.1 Authentifizierung in AWS

Die Authentifizierung in AWS erfolgt auf mehreren Ebenen, um die größt mögliche Sicherheit zu gewährleisten. Für die CI/CD-Pipeline werden spezielle IAM-Rollen für Jenkins, oder GitLab-Runner konfiguriert, die temporäre STS-Tokens

für Ephemeral Environments erhalten. Die lokale Entwicklung erfolgt über die AWS CLI, mit benutzerbasierten Zugriffsschlüsseln. Für Produktiv-Accounts ist Multi-Faktor-Authentifizierung (MFA) obligatorisch. Sensible Daten werden zentral im AWS Secrets Manager verwaltet [Amazon Web Services, 2025b]. Die Sicherheit wird durch das Least-Privilege-Prinzip in den IAM-Policies [Amazon Web Services, 2025c] sowie durch automatische Schlüsselrotation [Amazon Web Services, 2025e] gewährleistet.

3.2 Provisionierungs-Workflow

Der Provisionierungs-Workflow ist vollständig automatisiert und beginnt mit einem Code-Commit im Git-Repository, wodurch eine Pipeline in Jenkins oder GitLab CI ausgelöst wird [Jenkins, 2025, GitLab, 2025]. Die Terraform-Konfigurationen werden initialisiert, ein Plan erstellt und automatisierte Tests, insbesondere Security-Checks mit Checkov, durchgeführt [HashiCorp, 2025f, Bridgecrew, 2025]. Für die Produktivumgebung ist eine manuelle Freigabe erforderlich, nach dieser wird der Terraform-Apply-Prozess ausgeführt. Der Infrastrukturzustand wird in einem S3-Bucket mit Locking-Mechanismus gespeichert, um parallele Änderungen zu verhindern [HashiCorp, 2025c].

4 Vor-/Herausforderungen

4.1 Vorteile

Die gewählte Architektur bietet zahlreiche Vorteile, durch die Verwendung von Infrastructure-as-Code werden identische Umgebungen gewährleistet, was die Konsistenz über verschiedene Deployment-Stages hinweg sicherstellt [Amazon Web Services, 2025d]. Die vollständige Provisionierung kann in weniger als zehn Minuten abgeschlossen werden, was die Geschwindigkeit der Entwicklung und des Deployments erheblich steigert [HashiCorp, 2025f]. Die Kostentransparenz wird durch konsequentes Tagging aller Ressourcen gewährleistet [Amazon Web Services, 2025f]. Die Sicherheit profitiert von der versionierten Konfiguration, die Compliance-Anforderungen erfüllt [Open Policy Agent, 2025].

4.2 Herausforderungen

Trotz der vielen Vorteile gibt es auch Herausforderungen zu bewältigen, die Terraform-Syntax und die AWS-Services erfordern eine gewisse Einarbeitungszeit. Das Zustandsmanagement erfordert ein sorgfältiges Handling der state-Dateien, um Konflikte zu vermeiden [HashiCorp, 2025d]. Die Fehlersuche bei Abhängigkeitsproblemen kann komplex sein und erfordert ein tiefes Verständnis der Infrastruktur-Komponenten.

5 Technische Umsetzung

5.1 Terraform-Modulstruktur

Die Terraform-Konfiguration ist in logische Module aufgeteilt, die jeweils spezifische Aspekte der Infrastruktur verwalten, diese Module umfassen Netzwerk, Datenbank, Backend, Frontend und Sicherheit. Diese Struktur ermöglicht eine klare Trennung der Zuständigkeiten und erleichtert die Wartung und Erweiterung der Infrastruktur [HashiCorp, 2025b].

5.2 Beispiel: DynamoDB-Provisionierung

Ein konkretes Beispiel für die Provisionierung ist die DynamoDB-Tabelle für die To-Do-Items. Die Tabelle wird mit Pay-per-Request-Billing konfiguriert, was eine kosteneffiziente Skalierung ermöglicht. Die ID wird als Hash-Key definiert, und die Tabelle wird mit entsprechenden Tags für die Umgebung und das Management versehen.

Die automatisierte Provisionierung mit Terraform ermöglicht reproduzierbare, sichere und kosteneffiziente AWS-Umgebungen für die To-Do-Applikation. Durch modularen Code und CI/CD-Integration wird der Entwicklungslebenszyklus optimiert.

Literaturverzeichnis

- [Amazon Web Services, 2024] Amazon Web Services (2024). Understanding serverless architectures.
- [Amazon Web Services, 2025a] Amazon Web Services (2025a). Aws cloudformation documentation.
- [Amazon Web Services, 2025b] Amazon Web Services (2025b). Aws secrets manager documentation.
- [Amazon Web Services, 2025c] Amazon Web Services (2025c). Iam best practices - principle of least privilege.
- [Amazon Web Services, 2025d] Amazon Web Services (2025d). Infrastructure as code (iac) - aws well-architected framework.
- [Amazon Web Services, 2025e] Amazon Web Services (2025e). Rotating access keys - aws identity and access management.
- [Amazon Web Services, 2025f] Amazon Web Services (2025f). Tagging aws resources.
- [Bridgecrew, 2025] Bridgecrew (2025). Checkov documentation.
- [GitLab, 2025] GitLab (2025). Gitlab ci/cd documentation.
- [HashiCorp, 2025a] HashiCorp (2025a). Configuration language (hcl).
- [HashiCorp, 2025b] HashiCorp (2025b). Module best practices — terraform by hashicorp.
- [HashiCorp, 2025c] HashiCorp (2025c). Remote state management - terraform with amazon s3.
- [HashiCorp, 2025d] HashiCorp (2025d). State - purpose, pitfalls, and best practices.
- [HashiCorp, 2025e] HashiCorp (2025e). Terraform registry: Aws provider.
- [HashiCorp, 2025f] HashiCorp (2025f). What is terraform?
- [Jenkins, 2025] Jenkins (2025). Jenkins user documentation.
- [Open Policy Agent, 2025] Open Policy Agent (2025). Open policy agent: Terraform example.
- [Pulumi, 2025] Pulumi (2025). Pulumi documentation.